

BLINKPAY: AN OFFLINE NFC-BASED PAYMENT AND COMMUNICATION APPLICATION

A PROJECT BASED LEARNING (PBL) REPORT

Submitted

by

VISHAL KARTHIKEYAN K (2104251041094)

JAISOORYA K (2104251040327)

Submitted in partial fulfilment of the requirements

for the

Project-Based Learning component of Java Programming

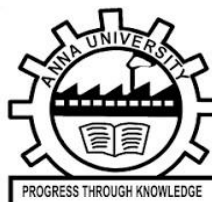
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY



(Autonomous)

Affiliated to Anna University, Chennai

October - 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and in still Scientifics park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R&D establishments.
- IM5:** To nurture incubation centers enabling structured entrepreneurship and start-ups

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

- DM1.** To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.
- DM2.** To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.
- DM3.** To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.
- DM4:** To enhance the fabric of research in computing through collaborative linkages with industry and academia.
- DM5:** To inculcate learning of the emerging technologies to pursue higher studies

PROGRAM EDUCATIONAL OBJECTIVES (PEO):

PEO 1: Contribute to the industry as an Engineer through sound knowledge acquired in core engineering to develop new processes and implement the solutions for industrial problems.

PEO 2: Establish an organization / industry as an Entrepreneur with professionalism, leadership quality, teamwork, and ethical values to meet the societal needs.

PEO 3: Create a better future by pursuing higher education / research and develop the sustainable products / solutions to meet the demand.

PROGRAM OUTCOMES (POS):

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOS):

PSO 1 : Analyze, design and develop solutions in the areas of Business Process Management to build the quality products for industry and social needs.

PSO 2 : Innovate ideas and solutions for real time problems in the field of Software Engineering and Mobile applications by adapting the emerging technologies and tools.

CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI

(Autonomous)

Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled “**BLINKPAY: AN OFFLINE NFC-BASED PAYMENT AND COMMUNICATION APPLICATION**” is a Bonafide record of work carried out by **VISHAL KARTHIKEYAN K (2104251041094)** and **JAISOORYA K (2104251040327)** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Java Programming course during the academic year 2026–2027 under my supervision.

SIGNATURE

Dr. S. Pavithra M.E., Ph. D

HEAD OF THE DEPARTMENT

Professor and Head,
Department of Computer Science and
Engineering
Chennai Institute of Technology
Chennai - 69

SIGNATURE

Dr. R. KAVITHA M.E., Ph. D

SUPERVISOR

Associate Professor
Department of Computer Science
and Engineering
Chennai Institute of Technology
Chennai - 69

Submitted for the final review held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We jointly declare that the PBL report on **“BLINKPAY: AN OFFLINE NFC-BASED PAYMENT AND COMMUNICATION APPLICATION”** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **“ANNA UNIVERSITY CHENNAI”** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.

Signature



VISHAL KARTHIKEYAN K

JAI SOORYA K

Place:

Chennai Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI.P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E, Ph.D.**, for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.**, who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr.S.PAVITHRA M.E, Ph.D.**, for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator **Dr.R KAVITHA M.E., Ph.D., Associate Professor**. Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **Dr.R KAVITHA M.E., Ph.D.,Associate Professor** and **Ms. A. Kalpana M.E., Assistant Professor** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

VISHAL KARTHIKEYAN K

2104251041094

JAISOORYA K

2104251040327

ABSTRACT

BlinkPay is an Android-based digital payment and communication application developed using Java to provide a simple and flexible platform for financial transactions and user interaction. The system addresses the limitations of conventional payment applications by supporting both offline and online transaction workflows. The offline payment mechanism uses Near Field Communication (NFC) to establish communication between sender and receiver and transfer transaction information without requiring an active internet connection. Online wallet data and pending transactions can be synchronized with a Supabase backend when network connectivity is available. The application incorporates Google authentication, PIN-based transaction verification, and biometric authentication to improve account and payment security. It also provides transaction history, wallet management, profile settings, push notifications, and a real-time chat system with text, voice-message, and file-sharing support. The implemented system supports offline NFC-based transactions as well as online payment processing and cloud synchronization—and is designed for rapid transaction processing with a target response time of less than 0.5 seconds under suitable conditions. Overall, BlinkPay demonstrates how Android, NFC, cloud synchronization, and secure authentication can be combined into a unified payment application.

Keywords: BlinkPay, NFC Technology, Offline Payments, Biometric Security, Digital Wallet

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	ix
	LIST OF TABLES	xii
	LIST OF FIGURES	xiii
	LIST OF ABBREVIATIONS	xiv
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 DRIVING QUESTION	2
	1.3 OBJECTIVES	2
	1.4 SCOPE AND LIMITATIONS	3
2	CONCEPT EXPLORATION	4
	2.1 RELATED APPROACHES	4
	2.1.1 TRADITIONALLY JAVA APPLICATION APPROACH	4
	2.1.2 JAVA FRAMEWORK-BASED APPROACH	5
	2.2 SUMMARY TABLE	6
	2.3 WHAT THIS TOLD US	7
3	PROJECT PLANNING AND TEAM ORGANISATION	8
	3.1 WEEKLY PBL PROGRESS LOG	8
	3.2 REQUIREMENTS	9
	3.3 FEASIBILITY	10
4	ITERATIVE DESIGN AND DEVELOPMENT	11
	4.1 SYSTEM ARCHITECTURE	11
	4.2 BASELINE	12
	4.3 PROJECT REFINEMENT	12
	4.4 FINAL APPROACH	13
	4.5 TESTING AND EXECUTION	14
5	IMPLEMENTATION	16
	5.1 MODULE DESCRIPTION	16
	5.2 KEY CODE SNIPPETS	17
	5.2.1 PAYMENT INPUT VALIDATION	18
	5.2.2 PIN HASHING AND VERIFICATION	18
	5.2.3 NFC PAYMENT COMMUNICATION	19
	5.2.4 INTERNET AVAILABILITY CHECK	19
	5.3 USER INTERFACE / DEMO	20
6	RESULTS AND DISCUSSION	23
	6.1 EVALUATION METRICS	23

CHAPTER NO.	TITLE	PAGE NO.
	6.2 RESULTS ACROSS ITERATIONS	25
	6.3 DISCUSSION	26
	6.4 LIMITATIONS	27
7	TEAM REFLECTION AND LEARNING OUTCOMES	29
	7.1 INDIVIDUAL REFLECTIONS	29
	7.2 TEAM LEARNING	29
	7.3 COURSE OUTCOMES — EVIDENCE SUMMARY	30
8	CONCLUSION AND FUTURE SCOPE	32
	8.1 CONCLUSION	32
	8.2 FUTURE SCOPE	32
	REFERENCES	34
	APPENDIX	36
	A.1 FULL SOURCE CODE	36
	A.2 WEEKLY LOG AND MENTOR SIGN-OFFS	36
	A.3 SELF AND PEER ASSESSMENT	38

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of Related Approaches	6
3.1	Weekly PBL Progress Log	8
3.2	Hardware and Software Requirements	9
6.1	Test and Defect Outcomes Across Iterations	23
A.2	Complete Weekly PBL Progress Log	36
A.3	Self and Peer Assessment	38

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System Architecture of BlinkPay	11
5.1	Sign-in Screen	20
5.2	Wallet Screen	20
5.3	Money Screen	21
5.4	PIN Screen	21
5.5	Chat Screen	21
5.6	Conversation Screen	21
5.7	NFC Send Screen	22
5.8	NFC Receive Screen	22

LIST OF ABBREVIATIONS

Abbreviation	Full Form
PBL	Project-Based Learning
NFC	Near Field Communication
PIN	Personal Identification Number
API	Application Programming Interface
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
URL	Uniform Resource Locator
URI	Uniform Resource Identifier
FCM	Firebase Cloud Messaging
SHA-256	Secure Hash Algorithm 256-bit
OS	Operating System
SDK	Software Development Kit
IDE	Integrated Development Environment
JDK	Java Development Kit
OOP	Object-Oriented Programming
MIME	Multipurpose Internet Mail Extensions
ISO-DEP	ISO/IEC 14443-4 Data Exchange Protocol

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Digital payment applications have become an important part of everyday financial activities, enabling users to transfer, receive, and manage money through mobile devices. However, many payment applications depend on continuous internet connectivity and conventional authentication mechanisms. When network availability is poor or unavailable, users may face difficulties in completing transactions. In addition, payment and communication services are often provided through separate applications, requiring users to switch between multiple platforms for different activities.

BlinkPay was developed as a Java-based Android application to explore a unified approach to digital payments and communication. The application provides both offline and online payment functionality, with Near Field Communication (NFC) used to enable device-to-device communication for offline transactions. The application also includes a digital wallet, transaction history, secure authentication, profile management, and cloud synchronization. Google-based sign-in, Blink PIN verification, and biometric authentication are incorporated to provide multiple levels of access and payment security.

BlinkPay further extends the payment experience by integrating communication features within the same application. Users can exchange text messages, voice messages, and files, while push notifications provide timely alerts for incoming communication. By combining payment, authentication, synchronization, and communication features in one Android platform, BlinkPay aims to provide a simple and convenient mobile application for different transaction and communication requirements.

1.2 DRIVING QUESTION

Can we develop a Java-based Android application that provides secure and convenient payment and communication services while supporting transactions even when internet connectivity is unavailable?

This question was narrowed into the practical task of designing and developing BlinkPay using Java and object-oriented programming principles. The project focused on identifying and implementing the required modules for wallet management, offline NFC payment, online payment processing, authentication, transaction history, cloud synchronization, and communication. The application was then incrementally developed, integrated, tested, and refined to produce a functional Android-based platform.

1.3 OBJECTIVES

- To analyze the real-world limitations of conventional digital payment applications and identify the requirements for BlinkPay.
- To design and develop a Java-based Android application using appropriate object oriented programming concepts, classes, interfaces, and modular components.
- To implement an offline NFC-based payment mechanism for communication between sender and receiver devices.
- To integrate online payment processing, wallet management, cloud synchronization, transaction history, and user account services
- To implement secure authentication using Google Sign-In, Blink PIN, and biometric authentication, along with application security controls.
- To test, debug, and refine the application to verify its functionality, reliability, error handling, and overall user experience.

1.4 SCOPE AND LIMITATIONS

The scope of BlinkPay covers the development of a Java-based Android application that combines digital wallet functionality, offline NFC-based transactions, online payment processing, authentication, transaction history, cloud synchronization, and communication features. The application supports wallet balance management, sending and receiving payments, online wallet operations, user profiles, and transaction records. The communication module supports text messaging, voice messages, and file sharing, with push notifications for incoming messages.

The offline payment functionality is designed for compatible NFC-enabled Android devices and requires NFC communication between the participating devices. Online payment processing, cloud synchronization, realtime communication, file and voice-message services, and push notifications require internet connectivity. Biometric authentication depends on the availability of supported biometric hardware on the user's device. The current account authentication implementation supports Google Sign-In; Microsoft account authentication is not included in the present version.

The project is developed and evaluated in the Android environment as a functional academic application. It does not attempt to replace complete banking infrastructure, regulated banking networks, or large-scale financial settlement systems. Factors such as operation across every Android device configuration, large-scale deployment, universal currency settlement, and full integration with banking institutions are outside the scope of the current project.

CHAPTER 2

CONCEPT EXPLORATION

2.1 RELATED APPROACHES

2.1.1 Traditional Java Application Approach

The first approach explored was a conventional Java-based application structure using object oriented programming principles such as classes, objects, inheritance, interfaces, and packages. Java's object-oriented model provides a structured way to represent application components as classes and define their interactions through interfaces [1]. The Java Collections Framework further provides reusable data structures and interfaces for managing groups of objects [2]. These concepts were relevant to BlinkPay because the application contains multiple independent components that can be organized into separate Java classes and modules.

For the Android implementation, native Android APIs were considered for device-specific functionality rather than using a general-purpose desktop Java application structure. Android's NFC framework provides NfcAdapter and reader-mode functionality [3], [4], while IsoDep provides ISO-DEP communication and transceive() operations for exchanging data [5]. This approach provided the low-level device access required for BlinkPay's offline payment mechanism.

The key outcome of this exploration was that a Java-based native Android architecture was suitable for implementing BlinkPay's core functionality. The team therefore selected Java classes and Android activities as the basic application structure and used native Android APIs for NFC, authentication, and device-level operations.

2.1.2 Java Framework-Based Approach

A second approach considered was a framework-oriented architecture in which a Java application depends on higher-level frameworks and external services rather than implementing every function directly. In conventional enterprise Java development, frameworks such as Spring Boot can be used for application services and REST-based backend systems. For BlinkPay, however, the project required an Android client with direct access to mobile hardware, so a server-oriented framework was not selected as the primary application framework.

Instead, the project adopted Android platform APIs together with specialized cloud and payment services. Android's BiometricPrompt provides a system-managed biometric authentication interface [6], while Credential Manager provides the Sign in with Google flow used by the application.

Cloud-based services were explored for functions that are unsuitable for purely local processing. Supabase Realtime supports subscriptions to database changes and enables realtime communication, making it suitable for BlinkPay's chat and cloud synchronization requirements [7]. Firebase Cloud Messaging provides notification and data-message delivery to Android applications and was selected for chat notifications [8].

Online payment integration was also explored through the Razorpay payment gateway. The payment flow uses an order before checkout and requires payment-signature verification to confirm the authenticity of the returned payment information [9]. This provided the required infrastructure for BlinkPay's online wallet payment functionality without requiring the application to implement payment processing itself.

The key outcome of this exploration was that a hybrid architecture was more appropriate for BlinkPay: Java and native Android APIs for the mobile application, combined with specialized services for cloud synchronization, realtime communication, notifications, and online payments.

2.2 SUMMARY TABLE

Table 2.1 summarises the two approaches reviewed in Section 2.1 and what each contributed to the design of BlinkPay. For each approach, it lists the references consulted and the reported result, showing why a native Java-based Android approach was selected for the application core while specialized services were integrated for cloud synchronization, communication, notifications, and online payments.

Table 2.1 Summary of Explored Approaches

Ref.	Approach	Reported Result
[1]	Java Object-Oriented Programming	Provides classes, objects, inheritance, interfaces, and packages for structured application development.
[2]	Java Collections Framework	Provides reusable interfaces, implementations, and algorithms for managing collections of objects.
[3], [4]	Android NFC Reader Mode	Provides controlled NFC reader operation for foreground Android activities.
[5]	Android IsoDep	Provides ISO-DEP communication and transceive() operations for exchanging NFC data.
[6]	BiometricPrompt and Credential Manager	Provides system-managed biometric authentication and a standardized Google Sign-In flow.
[7]	Supabase Realtime	Supports database-change subscriptions and realtime communication for connected clients.
[8]	Firebase Cloud Messaging	Provides notification and data-message delivery to Android applications.
[9]	Razorpay Payment Gateway	Provides order-based checkout and payment-signature verification for online payments.

2.3 WHAT THIS TOLD US

The exploration showed that BlinkPay required more than a conventional standalone Java application. The team selected a native Android Java approach for Iteration 1 because it provides direct access to Android functionality such as NFC and biometric authentication while allowing the application to be organized using Java classes and interfaces.

As development progressed, the application was extended using a hybrid service-based architecture, with Supabase for cloud data and realtime communication, Firebase Cloud Messaging for notifications, and Razorpay for online payment processing [7]–[9]. This approach allowed BlinkPay to retain a Java-based Android core while integrating the specialized services required by the complete application.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 WEEKLY PBL PROGRESS LOG

The development of BlinkPay was organized as an iterative Project-Based Learning process. The project was divided into stages covering problem identification, objective setting, technical exploration, system design, implementation, testing, demonstration, and final documentation. Each stage was used to progressively develop and refine the application based on project progress and mentor feedback.

Table 3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done
1–2	Problem Statement, Objectives, Time Plan and Technical Background	Identified the problem of dependence on internet connectivity in conventional payment systems. Defined BlinkPay as an Android application combining offline NFC payments, online payment functionality, digital wallet features, and communication services. Prepared the initial objectives and project plan.
3-4	Requirements Analysis and Methodology Design	Explored NFC communication, authentication mechanisms, cloud synchronization, realtime communication, push notifications, and online payment integration. Designed the overall module structure and selected Java and Android as the primary development platform.
5-7	Implementation and Testing	Developed the core Android application, wallet interface, payment amount validation, PIN and biometric authentication, NFC sender/receiver workflow, local wallet handling, and transaction history. Performed initial functional testing and debugging.

Week	Milestone / Task	Work Done
8–10	Feature Integration and Presentation with Demo	Integrated cloud-based user and wallet services, synchronization, online wallet deposits, realtime chat, voice messages, file sharing, and push notifications. Prepared and demonstrated the progressively integrated application.
11-12	Integrated cloud-based user and wallet services, synchronization, online wallet deposits, realtime chat, voice messages, file sharing, and push notifications. Prepared and demonstrated the progressively integrated application.	Refined the NFC payment workflow, authentication, application security, synchronization, transaction history, communication features, and user interface. Performed final functional testing, corrected identified issues, and prepared the project report and final demonstration.

3.2 REQUIREMENTS

BlinkPay is implemented as a Java-based Android application. Unlike a conventional server side Java project, it does not use JDBC, Spring Boot, Hibernate, or a direct MySQL/PostgreSQL connection from the Android application. Instead, Android APIs are used for device functionality, while cloud and external services are accessed through their respective APIs and SDKs.

Table 3.2 Development Requirements

Category	Requirement
Processor / RAM	Development computer capable of running Android Studio and the Android build environment.
Programming language	Java
Android Platform	Android SDK / Android operating system
Libraries / frameworks	AndroidX, Android Biometric APIs, Android NFC APIs
Networking	OkHttp
Data Handling	JSON
Backend / Database	Supabase
Notifications	Firestore Cloud Messaging
Online Payment	Razorpay Android SDK

Development Environment	Android Studio
Testing Device	NFC-enabled Android smartphone; biometric hardware where applicable

The source code uses AndroidX and Android platform APIs for application activities and biometric/NFC functionality, OkHttp for network communication, JSON for data exchange, Firebase Cloud Messaging for notifications, and Razorpay's Android SDK for online payment processing.

Supabase is used for backend operations including user, wallet, transaction, and synchronization-related data.

3.3 FEASIBILITY

BlinkPay was feasible within the PBL timeframe because the project could be developed incrementally using the Android platform and existing APIs and services. The application was divided into manageable modules, allowing the core wallet and NFC payment functionality to be developed first and additional features such as authentication, synchronization, online payment processing, realtime communication, file sharing, and notifications to be integrated progressively. The use of established Android APIs and external services reduced the need to implement low-level communication, notification, and payment infrastructure from the beginning. This modular and iterative approach allowed the team to test individual functions, identify issues, and refine the application throughout the PBL development cycle.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 SYSTEM ARCHITECTURE

User Input → Validation → Java Processing → Data Handling → Output

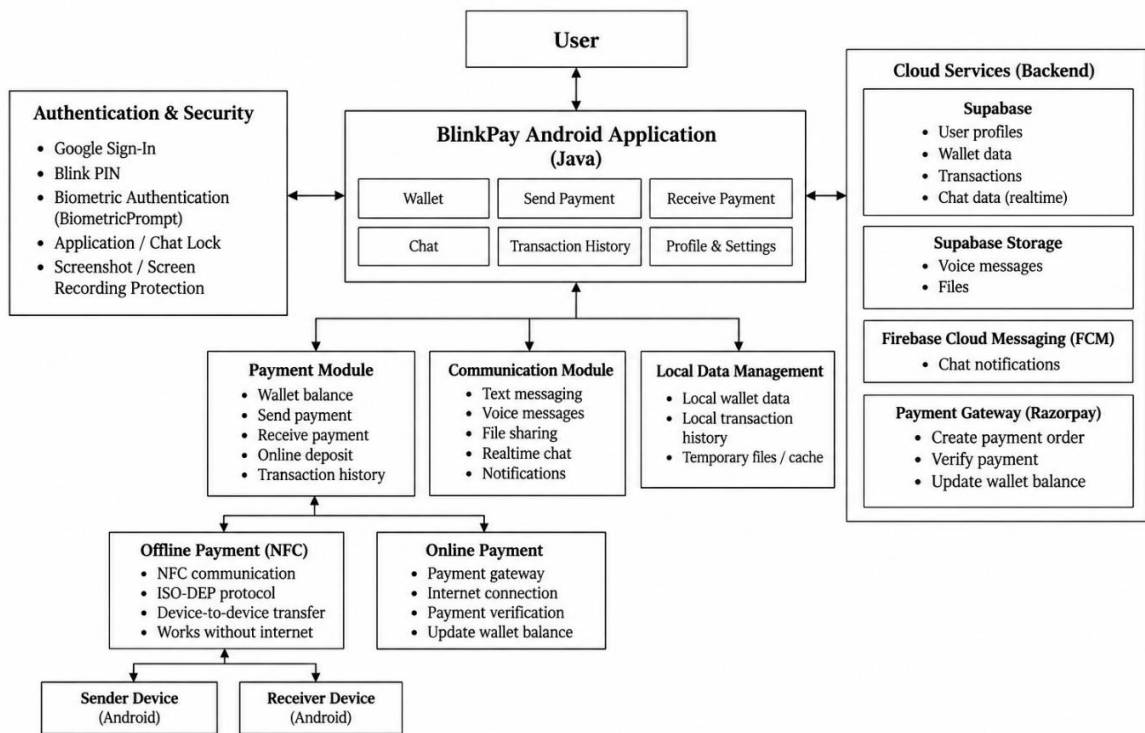


Figure 4.1 System Architecture

Figure 4.1 System Architecture of BlinkPay

For BlinkPay, the above flow is extended to accommodate the application's payment, security, communication, and cloud-based functionality. User actions such as entering a payment amount, authenticating an account, initiating an NFC transaction, sending a message, or performing a wallet operation are first received through the Android user interface. The application then validates the input before passing it to the appropriate Java class or activity for processing.

The processed information is handled either locally or through external services depending on the operation. Local application data includes wallet balance, preferences, and transaction history, while cloud-based operations use backend services for synchronization, communication, and online payment processing.

The final result is presented to the user through the appropriate Android interface, such as a payment confirmation, updated wallet balance, transaction record, or chat message.

4.2 BASELINE

The first version of BlinkPay concentrated on implementing the fundamental Android application and the basic payment workflow. The initial structure consisted of separate Java activities for major operations such as the wallet, amount entry, payment authentication, and NFC communication. The application used classes, objects, methods, conditional statements, and event-driven callbacks to control user interaction and application flow.

The initial payment process required the user to enter an amount and pass validation before continuing to the authentication stage. The application checked that the entered amount was valid and did not exceed the available wallet balance. After validation, the application proceeded to the payment authentication stage and subsequently to NFC communication.

The baseline NFC implementation used Android's NfcAdapter, Tag, and IsoDep APIs to establish communication between the sender and receiver devices. The sender could connect to the receiver, exchange payment information, process the response, and update the local wallet and transaction history after successful completion.

Although the baseline provided the core payment functionality, the initial implementation had a more limited feature set. Additional work was therefore required to improve validation, authentication, synchronization, communication, notification handling, and overall modularity.

4.3 PROJECT REFINEMENT

The application was progressively refined by separating functionality into specialized Java classes and Android activities. Instead of keeping all operations in a single component, payment, authentication, NFC

communication, cloud synchronization, chat, notifications, and profile settings were implemented as separate functional modules. This modular structure reduced the coupling between major operations and made individual components easier to develop and test.

Object-oriented programming concepts were applied during the refinement process. Data and operations were encapsulated within appropriate classes, while inheritance was used through Android activity and service classes. Interface-based callbacks were also used for operations such as NFC reader callbacks, biometric authentication callbacks, payment-result handling, and asynchronous backend operations. For example, `SendNfcActivity` implements `NfcAdapter.ReaderCallback`, while payment processing uses `PaymentResultWithDataListener`.

Exception handling was incorporated to manage invalid input, failed network requests, invalid responses, authentication failures, and NFC communication errors. The payment amount module validates empty, invalid, zero, and insufficient-balance conditions before continuing.

The refinement stage also introduced cloud synchronization and communication functionality. The synchronization module checks internet availability and tracks transactions that require synchronization. The communication module was extended to support realtime messaging, voice messages, and file sharing, while Firebase Cloud Messaging was used for notifications.

4.4 FINAL APPROACH

The final BlinkPay application uses a modular and object-oriented Java design in which each major functionality is handled by dedicated classes and Android components. Encapsulation is used to keep related data and operations within individual modules, while inheritance is provided through Android component classes such as activities and services. Interfaces and callback mechanisms

allow different modules to communicate with Android frameworks and external services without tightly coupling their implementations.

Abstraction is achieved by separating complex operations into helper and service classes. For example, backend operations are organized within SupabaseHelper, while synchronization operations are separated into SyncHelper. This allows application activities to request operations without containing all of the underlying networking and data-handling logic themselves.

Polymorphism is reflected through the use of overridden lifecycle and callback methods provided by Android and other implemented interfaces. The project also uses collection and structured data-handling mechanisms where required for transaction, chat, and synchronization data. JSON objects and arrays are used to represent and process structured application information.

The final design therefore combines Java object-oriented concepts with Android platform functionality. NFC is used for offline payment communication, biometric and PIN mechanisms are used for authentication, cloud services support synchronization and communication, Firebase provides notifications, and the online payment gateway handles online payment processing.

4.5 TESTING AND EXECUTION

The BlinkPay application was tested at the module and workflow levels to verify compilation, functionality, input validation, exception handling, navigation, and output behavior. Each major feature was executed through its expected user flow and checked for both successful and invalid conditions.

The payment amount module was tested for valid amounts, empty input, invalid numeric values, zero or negative values, and insufficient wallet balance. Only valid values within the available balance are allowed to proceed to authentication.

The authentication workflow was tested for PIN and biometric authentication. PIN verification validates the required four-digit input and compares its SHA-256 hash with the stored value. When biometric authentication is enabled and succeeds, the application proceeds to the NFC stage.

The NFC transaction workflow was tested through the sender and receiver stages. The application checks NFC availability, establishes reader mode, detects the target device, connects through ISO-DEP, exchanges protocol messages, validates responses, and handles communication errors.

Other application modules were tested for cloud synchronization, transaction-history handling, realtime chat, voice-message and file operations, and push notifications. The synchronization module explicitly checks for internet availability before attempting cloud synchronization.

The final testing stage focused on verifying that the integrated modules operate together correctly and that invalid inputs or failed operations are handled without interrupting the overall application flow.

CHAPTER 5

IMPLEMENTATION

5.1 MODULE DESCRIPTION

The BlinkPay Android application is implemented using Java and is divided into multiple functional modules. This modular structure keeps payment, authentication, data management, communication, and user-interface operations organized and allows individual components to be developed and tested independently.

- **Input Module:** The Input Module collects information entered by the user through the Android interface. Examples include account information during authentication, the payment amount entered before sending money, PIN input for transaction verification, and content entered in the communication module. The payment amount is read from the input field and passed to the next stage only after basic validation.
- **Processing Module:** The Processing Module performs the main application operations using Java classes, methods, conditions, callbacks, and Android APIs. In the payment workflow, the entered amount is processed, the local wallet balance is checked, authentication is performed, and the transaction is then passed to the NFC communication stage. The NFC module uses NfcAdapter, Tag, and IsoDep to establish communication and exchange payment information with the receiving device.
- **Data Management Module:** The Data Management Module manages local and cloud-based application data. Local information such as wallet balance, user preferences, and transaction history is maintained using Android application storage, while cloud-based user, wallet, transaction, and synchronization operations are handled through Supabase. Transaction history is represented using JSON objects and

arrays, and synchronization functionality checks for pending records before updating cloud data.

- **Validation and Security Module:** The Validation and Security Module ensures that invalid input and unauthorized operations are rejected. The application validates payment amounts, checks for sufficient wallet balance, 26 verifies four-digit PIN input, and handles authentication failures. PIN values are processed using SHA-256 hashing, while biometric authentication is implemented using Android's BiometricPrompt.
- **Communication Module:** The Communication Module provides text messaging, realtime communication, voice messages, and file sharing. Realtime chat events are received and processed through the communication subsystem, while voice messages and shared files are handled together with associated metadata such as file name, MIME type, and file size.
- **Output Module:** The Output Module presents processed information to the user through Android screens, notifications, status messages, wallet information, transaction history, and chat interfaces. After a successful payment, the application updates the payment status and wallet information, while notification handling can direct the user to the relevant chat interface.

5.2 KEY CODE SNIPPETS

Only the most important Java implementation snippets are included below. The complete source code of the application is provided separately as described in Appendix A.1.

Code 5.2.1 Payment Input Validation

```
private void continueToPin() {
    String amountText =
        amountInput.getText()
            .toString()
            .trim();
    if (amountText.isEmpty()) {
        amountInput.setError("Enter amount");
        return;
    }
    double amount;
    try {
        amount = Double.parseDouble(amountText);
    } catch (Exception e) {
        amountInput.setError("Invalid amount");
        return;
    }
    if (amount <= 0) {
        amountInput.setError(
            "Amount must be greater than 0"
        );
        return;
    }
    float balance =
        getSharedPreferences(
            "BlinkPayPrefs",
            MODE_PRIVATE
        ).getFloat(
            "wallet_balance",
            0.00f
        );
    if (amount > balance) {
        Toast.makeText(
            this,
            "Insufficient balance",
            Toast.LENGTH_SHORT
        ).show();
        return;
    }
}
```

Code 5.2.2 PIN Hashing and Verification

```
MessageDigest digest =
    MessageDigest.getInstance("SHA-256");
byte[] hash =
    digest.digest(
        pin.getBytes(
            java.nio.charset.StandardCharsets.UTF_8
        )
    );
```


Code 5.2.3 NFC Payment Communication

```
@Override
public void onTagDiscovered(Tag tag) {
    IsoDep isoDep =
        IsoDep.get(tag);
    if (isoDep == null) {
        return;
    }
    try {
        isoDep.connect();
        isoDep.setTimeout(5000);
        byte[] response =
            isoDep.transceive(
                NfcProtocol.HELLO.getBytes(
                    StandardCharsets.UTF_8
                )
            );
        // Process NFC response
    } catch (Exception e) {
        // Handle NFC communication failure
    }
}
```

Code 5.2.4 Internet Availability Check

```
public static boolean hasInternet(Context context) {
    ConnectivityManager cm =
        (ConnectivityManager)
            context.getSystemService(
                Context.CONNECTIVITY_SERVICE
            );
    if (cm == null) {
        return false;
    }
    Network network =
        cm.getActiveNetwork();
    return network != null;
}
```

The complete source code is referenced in Appendix A.1.

5.3 USER INTERFACE / DEMO

Figures 5.1 to 5.8 show screenshots of the running BlinkPay Android application, illustrating the sign-in, wallet, payment, PIN authentication, chat, and related application interfaces.

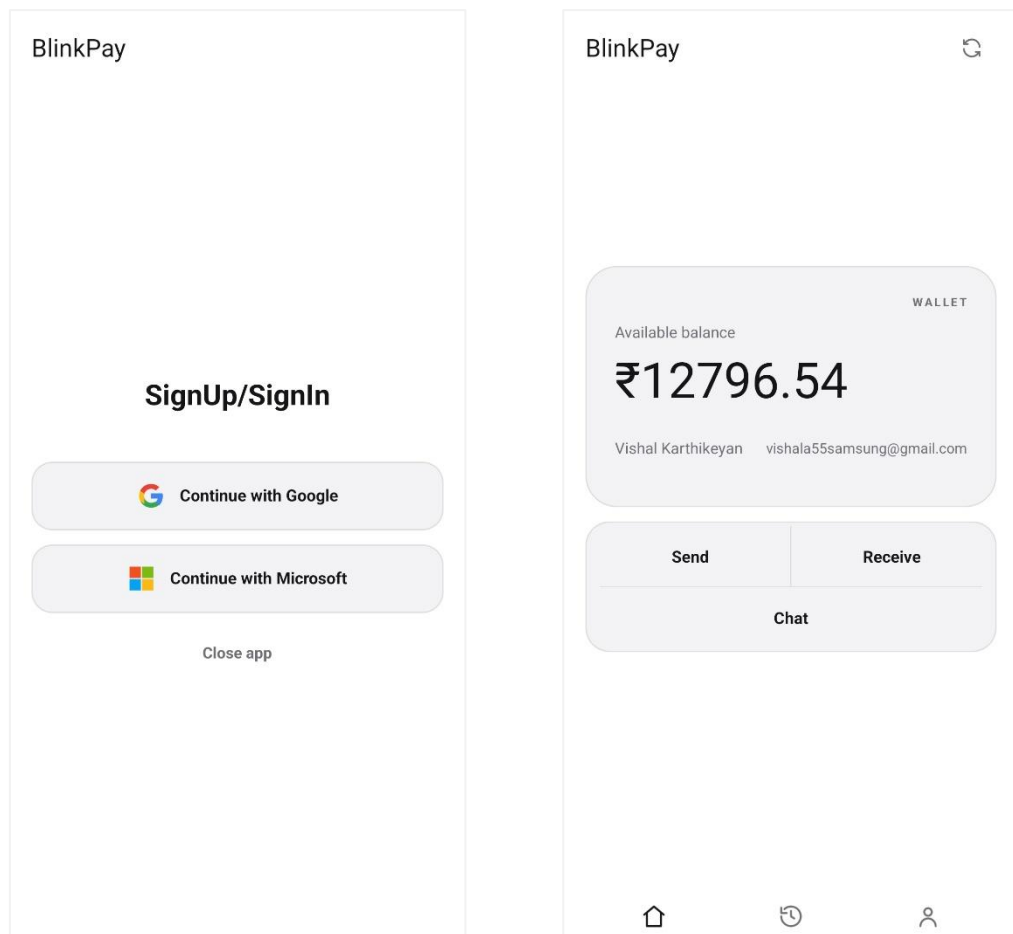


Figure 5.1, 5.2 – Sign-in Screen and Wallet Screen

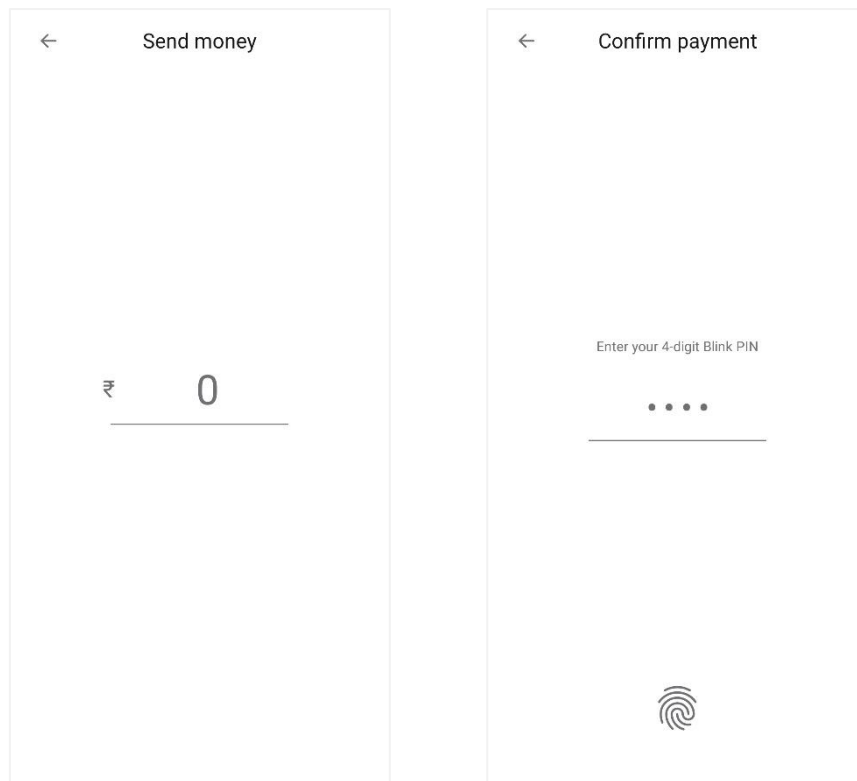


Figure 5.3, 5.4 – Money Screen and PIN Screen

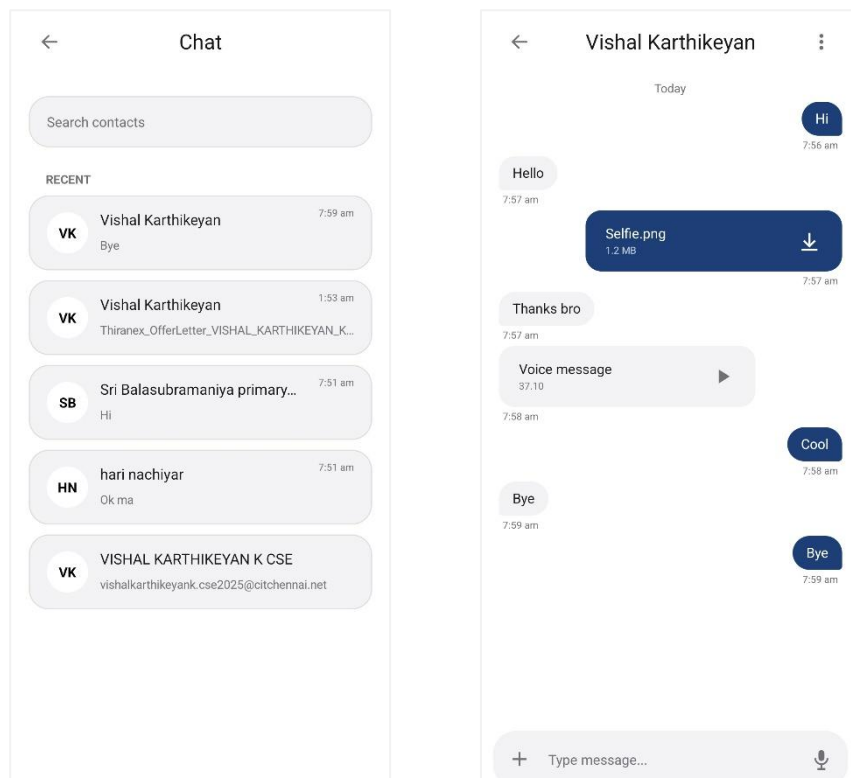


Figure 5.5, 5.6 – Chat Screen and Conversation Screen

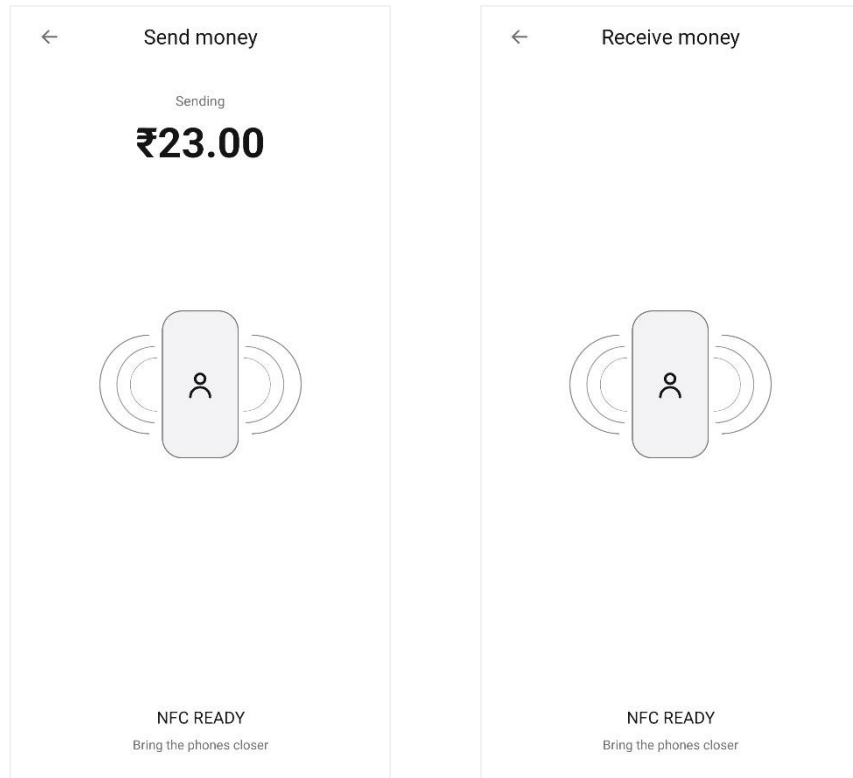


Figure 5.7, 5.8 – NFC Send Screen and Receive Screen

CHAPTER 6

RESULTS AND DISCUSSION

6.1 EVALUATION METRICS

The BlinkPay Android application was evaluated using metrics that are appropriate for a Java based mobile application. The evaluation focused on functional correctness, transaction reliability, application response, authentication, synchronization, and defects identified during successive development iterations.

Test Case Pass Rate

The test case pass rate was used to measure the functional correctness of the application. Test cases were prepared for major modules such as wallet operations, payment amount validation, PIN authentication, biometric authentication, NFC communication, online payment processing, transaction history, synchronization, chat, file transfer, and voice communication.

The pass rate was calculated as:

Test Case Pass Rate (%) = (Number of Passed Test Cases / Total Number of Test Cases) × 100

A higher pass rate indicates that the implemented functions are behaving according to the expected requirements.

NFC Transaction Success Rate

Since offline NFC payment is a major feature of BlinkPay, NFC transaction success rate was considered separately. Tests were performed for sender-receiver pairing, NFC detection, connection establishment, data transmission, and successful completion of the payment communication.

This metric is important because NFC communication depends on both compatible devices and correct handling of connection and transmission failures.

Average Response Time

Average response time was used to examine the responsiveness of major application operations. The measurement can include operations such as screen transitions, amount validation, authentication processing, NFC communication stages, and online payment processing

Average Response Time = Total observed execution time / Number of observations

This metric helps determine whether the application provides a responsive user experience.

Authentication Success Rate

Authentication testing covered the Blink PIN and biometric authentication mechanisms implemented in the application. Test cases included valid authentication, invalid PIN entry, incorrect PIN length, and biometric authentication outcomes.

This metric was selected because authentication is directly related to protecting payment operations.

Synchronization Success Rate

BlinkPay supports synchronization of locally stored transaction information with the cloud service when network connectivity is available. Therefore, synchronization success rate was used to evaluate whether pending transaction data was successfully transferred during synchronization.

Defects Identified Across Iterations

The number of defects identified and corrected during baseline development, project refinement, and final implementation was also recorded. This provides evidence of iterative improvement and shows how testing contributed to the refinement of the application.

Code Coverage

Code coverage may be included when automated Java/JUnit tests are available. It indicates the proportion of program code executed by the test suite. The actual code-coverage percentage should be entered only from the generated coverage report. Manual testing results should not be presented as code coverage.

6.2 RESULTS ACROSS ITERATIONS

The application was developed incrementally through three major stages: Baseline, Project Refinement, and Final Approach. Testing was performed during each stage so that functional issues and missing requirements could be identified and corrected before proceeding to the next iteration.

Table 6.1 Test and Defect Outcomes Across Iterations

Version	Tests Cases Passed	Code Coverage	Average Response Time
Baseline (Iteration 1)	—	—	—
Project Refinement (Iteration 2)	—	—	—
Final Approach (Iteration 3)	19 / 19	—	0.412s

The baseline version focused on the basic payment workflow, including wallet operation, amount entry, authentication, and NFC-based payment communication. Initial testing was used to identify functional problems and missing validation conditions.

During the project refinement stage, additional modules and error-handling mechanisms were integrated. These included improved input validation, local transaction handling, cloud synchronization, communication features, and notification-related functionality. Testing at this stage focused on module interaction and handling of invalid or exceptional inputs.

In the final approach, the integrated application was tested as a complete system. The evaluation covered payment workflows, authentication, NFC communication, online payment processing, transaction history,

synchronization, chat functionality, file and voice communication, and application navigation.

6.3 DISCUSSION

The evaluation shows how the BlinkPay application evolved through iterative development. The baseline implementation established the fundamental payment workflow and provided the foundation for subsequent improvements. The main focus at this stage was to verify that the essential operations could be completed correctly.

One of the important improvements during refinement was stronger input validation. Payment amount validation checks whether the entered amount is present, numerically valid, greater than zero, and within the available wallet balance. These checks reduce invalid transaction attempts and improve reliability before authentication and NFC communication are initiated.

The authentication workflow was also strengthened during development. BlinkPay supports a four-digit Blink PIN using SHA-256 hashing and also provides biometric authentication through Android biometric APIs. This provides multiple authentication paths while maintaining authentication before the NFC payment operation.

The NFC implementation was another important part of the iterative development process. The application uses Android NFC APIs including `NfcAdapter`, `Tag`, and `IsoDep` to establish communication between devices. The communication process includes connection handling, timeout configuration, data transmission, and exception handling. These mechanisms are important because NFC communication can fail when the devices are moved apart, when the required technology is unavailable, or when communication is interrupted.

The data-management functionality was expanded beyond the initial local-wallet implementation. Local transaction information is maintained by the

application and synchronization support is provided when network connectivity is available. This allows the application to maintain local transaction information while also supporting cloud-based synchronization.

The communication functionality further expanded the scope of the application from payment processing to integrated messaging. The chat implementation supports text messages as well as file and voice-related communication. This required handling different message types and associated file information in addition to normal text communication.

Overall, the iterative process helped transform the initial payment-focused implementation into a more complete Android application. Each stage introduced additional functionality while testing was used to detect errors, validate requirements, and improve the reliability of the previous implementation.

6.4 LIMITATIONS

The evaluation of BlinkPay has several limitations. First, the testing was performed within the available project-development environment and therefore may not represent the behavior of the application across all Android devices, NFC hardware configurations, network conditions, and biometric implementations.

Second, the number of test cases may be limited compared with a production-scale financial application. Some rare edge cases, unusual NFC interruption scenarios, and unexpected user interactions may therefore remain untested.

Third, the project evaluation does not constitute large-scale multi-user or load testing. The application was evaluated primarily as a student project and not under the transaction volume expected from a large commercial payment platform.

Fourth, response-time measurements can vary depending on device hardware, Android version, NFC hardware, network conditions, and external service

availability. Therefore, measured performance should be reported together with the testing environment.

Fifth, automated code coverage should not be claimed unless a valid Java/JUnit coverage report was generated. Similarly, performance values should be included only when they were actually measured.

Finally, some planned functionality was outside the final implementation scope. In particular, Microsoft account authentication was not included in the implemented version. The final evaluation therefore covers only the features actually implemented and tested in the submitted application.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 INDIVIDUAL REFLECTIONS

VISHAL KARTHIKEYAN K: I worked on the development and integration of the BlinkPay Android application, with particular focus on implementing the Java-based application workflow and integrating payment, authentication, NFC communication, and supporting services. Through this project, I gained practical experience in applying Java and object-oriented programming concepts to a real Android application rather than using them only in academic examples. One major challenge was coordinating multiple components such as NFC communication, authentication, local data handling, and cloud services so that they worked together reliably.

JAISOORYA K: I contributed to the development, testing, integration, and refinement of the BlinkPay project across its different modules. This project helped me understand how individual Android components and external services can be combined to build a complete application with payment and communication functionality. A major challenge was identifying and resolving issues that occurred when different modules interacted, particularly during testing and final integration.

7.2 TEAM LEARNING

The BlinkPay project provided practical experience in collaborative software development. The work was divided according to the different functional requirements of the application, and the implementation was progressively integrated into a single Android application. This approach allowed the team to develop individual components, test them, identify issues, and refine the overall

system rather than attempting to implement the complete application at one time.

The team learned that integration is as important as individual module development. Features such as payment validation, authentication, NFC communication, transaction history, synchronization, chat, notifications, and online payment processing depend on interaction between multiple components. Therefore, changes made in one part of the application had to be checked against the behavior of the related modules.

Another important learning outcome was the value of iterative testing. Problems were identified during implementation and addressed through validation, exception handling, 39 40 improved communication between modules, and repeated testing. This helped the team understand that software development is an incremental process in which design decisions may need to be refined as practical problems are discovered.

The project also improved communication and decision-making within the team. Technical decisions had to consider not only whether a feature could be implemented, but also whether it could be integrated reliably within the available time and project scope.

If the PBL cycle were restarted, the team would plan the integration and testing stages earlier and maintain more systematic test records from the beginning. The team would also document individual responsibilities and major design decisions throughout development so that progress and learning could be demonstrated more clearly during the final review.

7.3 COURSE OUTCOMES — EVIDENCE SUMMARY

Evidence from this project for each outcome of the Java Programming course is summarised below.

CO1 – Object-oriented design: Applied Java programming and object-oriented concepts such as classes, objects, inheritance, interfaces, methods, and modular design while developing the BlinkPay Android application.

CO2 – Android Application Development: Used Java and Android platform APIs to implement application workflows including payment validation, authentication, NFC communication, wallet management, and user interaction.

CO3 – API and Service Integration: Integrated external services and Android components including Credential Manager, BiometricPrompt, Supabase, Firebase Cloud Messaging, and the Razorpay Android SDK for authentication, cloud synchronization, notifications, and online payment processing.

CO4 – Exception Handling and Validation: Applied input validation, exception handling, callback mechanisms, and error handling for invalid payment amounts, authentication failures, network problems, and NFC communication errors.

CO5 – Testing and Debugging: Performed iterative testing and debugging of payment, authentication, NFC, synchronization, communication, and notification workflows. The final implementation recorded 19/19 passing test cases and an observed average response time of 0.412 seconds.

The BlinkPay project provided learning beyond the implementation of the final application. The team gained experience in Java programming, Android development, API integration, debugging, testing, security-aware application design, and collaborative problem solving. The iterative nature of the PBL process helped connect theoretical concepts with practical software development and demonstrated the importance of planning, implementation, testing, refinement, and reflection throughout a project lifecycle.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

BlinkPay is a Java-based Android application developed to provide an integrated digital payment and communication experience. The application implements offline NFC-based transactions along with online payment processing, wallet management, transaction history, authentication, and cloud synchronization. Security features include Google authentication, Blink PIN verification, and biometric authentication, while the communication module supports real-time text messaging, voice messages, and file sharing. The project also integrates Android NFC APIs, Supabase services, Firebase Cloud Messaging, and the Razorpay Android SDK to provide the required application functionality. Through iterative development, testing, and refinement, the team developed and integrated the major modules into a unified application. The project therefore demonstrates the practical application of Java programming, Android development, object-oriented design, API integration, and software testing to address the defined project objectives.

8.2 FUTURE SCOPE

- Expanded authentication: Additional authentication providers and stronger authentication options can be integrated to provide more flexible account access.
- Improved NFC reliability: NFC communication can be further optimized for more device models, interrupted connections, and different real-world usage conditions.
- Advanced transaction management: Future versions can provide more detailed transaction tracking, improved synchronization handling, and additional wallet management capabilities.

- Enhanced communication: The chat module can be extended with features such as richer media sharing, improved voice communication, and additional messaging controls.
- Large-scale testing: The application can be evaluated using more devices, larger numbers of users, concurrency testing, and broader performance testing to identify issues that may not appear during prototype testing.
- Production-oriented deployment: Future development can focus on stronger backend infrastructure, expanded financial-service integration, monitoring, and deployment requirements for a larger-scale application.

REFERENCES

- [1] Oracle, “Object-Oriented Programming Concepts,” The Java Tutorials, Oracle Corporation. [Online]. Available: Oracle Java OOP Concepts. [Accessed: Sep. 22, 2026].
- [2] Oracle, “The Java Collections Framework,” The Java Tutorials, Oracle Corporation. [Online]. Available: Oracle Java Collections Framework. [Accessed: Sep. 22, 2026].
- [3] Google, “NfcAdapter,” Android Developers, Android API Reference. [Online]. Available: Android NfcAdapter API. [Accessed: Sep. 22, 2026].
- [4] Google, “NfcAdapter.ReaderCallback,” Android Developers, Android API Reference. [Online]. Available: Android ReaderCallback API. [Accessed: Sep. 22, 2026].
- [5] Google, “IsoDep,” Android Developers, Android API Reference. [Online]. Available: Android IsoDep API. [Accessed: Sep. 22, 2026].
- [6] Google, “BiometricPrompt,” Android Developers, AndroidX API Reference. [Online]. Available: AndroidX BiometricPrompt API. [Accessed: Sep. 22, 2026].
- [7] Supabase, “Realtime,” Supabase Documentation. [Online]. Available: Supabase Realtime Documentation. [Accessed: Sep. 22, 2026].
- [8] Firebase, “Firebase Cloud Messaging,” Firebase Documentation. [Online]. Available: Firebase Cloud Messaging Documentation. [Accessed: Sep. 22, 2026].

- [9] Razorpay, “Android Standard Checkout Integration,” Razorpay Documentation. [Online]. Available: Razorpay Android Documentation. [Accessed: Sep. 22, 2026].

APPENDIX

A.1 FULL SOURCE CODE

The complete source code of the BlinkPay Android application is provided as an accompanying source-code package. The package contains the Java source files and supporting project files used for application development, including authentication, wallet and payment processing, NFC communication, data synchronization, communication, and notification modules.

Source code Archive:

[BlinkPay.zip](#)

BlinkPay APK:

[BlinkPay.apk \(Signed\)](#)

A.2 WEEKLY LOG AND MENTOR SIGN-OFFS

The complete weekly PBL progress log is provided below. It records the major activities, development progress, testing, refinement, and documentation carried out during the project.

Week	Major Activity	Output	Mentor Remarks
1	Problem identification and project planning	Problem statement and objectives	Problem identified clearly and project objectives were defined.
2	Technical study and requirement analysis	Initial requirements	Technical requirements were studied and relevant technologies were identified.
3	Requirement analysis and methodology planning	Requirement specification	Requirements were organized and a suitable

			development approach was planned.
4	System architecture and module planning	Initial system design	System architecture and major application modules were planned appropriately.
5	Baseline Android application development	Initial application build	Baseline Android application was developed with the core payment workflow.
6	Authentication and payment workflow implementation	Authentication and validation modules	Authentication and payment validation modules were implemented and tested.
7	NFC payment implementation and testing	NFC sender/receiver functionality	NFC sender and receiver communication was implemented and tested successfully.
8	Feature integration and project refinement	Integrated application modules	Additional modules were integrated and the application was refined progressively
9	Cloud synchronization and communication implementation	Sync and chat functionality	Cloud synchronization and communication features were integrated into the application.

10	File, voice-message and notification integration	Extended communication features	Voice, file-sharing, and notification features were integrated and verified.
11	Final testing, debugging and corrections	Refined final build	Final testing and debugging were carried out and identified issues were corrected.
12	Final documentation and presentation preparation	Final application and report	Documentation was completed and the final application was prepared for presentation.

A.3 SELF AND PEER ASSESSMENT

The complete weekly PBL progress contribution of each team member to the project. Both members rated the work as an equal 50% share, and the remarks column summarises the modules each member was primarily responsible for.

Table A.1 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
VISHAL KARTHIKEYAN K	50%	50%	Contributed to Java development, payment workflow implementation, testing, debugging, and project documentation.
JAISOORYA K	50%	50%	Contributed to application development, module integration, testing, debugging, and project documentation.

